Interio

Веб-приложение для дизайна интерьера с системой аутентификации пользователей и квизом для подбора стиля.

Возможности

- **Система аутентификации** - регистрация, вход, управление сессиями
- **Интерактивный квиз** - подбор дизайна интерьера по предпочтениям
- **Админ панель** - управление заявками и пользователями
- **Адаптивный дизайн** - работает на всех устройствах
- **Auto keep-alive** - предотвращение "сна" на Render.com

Технологический стек

Backend

- **FastAPI** - современный Python фреймворк
- **SQLite** - база данных
- **Pydantic** - валидация данных
- **Uvicorn** - ASGI сервер

Frontend

- **Vanilla JavaScript** - без фреймворков
- **CSS3** - стилизация с анимациями
- **HTML5** - семантическая разметка

Деплой

- **Render.com** - хостинг
- **Python 3.11** - среда выполнения
- **Auto keep-alive** - поддержка активности

📁 Структура проекта

```
project-design/
├── server.py              # FastAPI приложение
├── requirements.txt       # Зависимости Python
├── render.yaml           # Конфигурация Render.com
├── start.bat             # Скрипт для локального запуска (Windows)
├── static/               # Статические файлы
│   ├── styles.css        # Стили приложения
│   ├── state-manager.js  # Управление состоянием
│   ├── auth.js          # Аутентификация
│   ├── quiz.js          # Логика квиза
│   └── admin.js         # Админ панель
├── templates/            # HTML шаблоны
│   ├── index.html       # Главная страница
│   ├── quiz.html        # Страница квиза
│   └── admin.html       # Админ панель
└── data.db              # База данных SQLite
```

🚀 Быстрый старт

Локальный запуск

1. **Клонируйте репозиторий** `bash git clone <repository-url> cd project-design`
2. **Установите зависимости** `bash pip install -r requirements.txt`
3. **Запустите приложение** ```bash # Windows start.bat

Linux/Mac python server.py # порт из PORT, по умолчанию 8000 ```

1. **Откройте в браузере** `http://localhost:8000`

Деплой на Render.com

1. **Подключите репозиторий к Render.com**
2. **Создайте Web Service**
3. **Используйте настройки из** `render.yaml`
4. **Установите переменные окружения:** - `APP_URL=https://your-app.onrender.com`

🔌 API Эндпоинты

Аутентификация

- `POST /api/auth/check-phone` - проверка телефона
- `POST /api/auth/login` - вход
- `POST /api/auth/register` - регистрация
- `POST /api/auth/logout` - выход
- `GET /api/auth/current-user` - текущий пользователь

Квиз

- `POST /api/quiz/submit` - отправка заявки

- `GET /api/quiz/submissions` - список заявок

Админ панель

- `POST /api/auth/admin-login` - вход в админку
- `GET /api/admin/submissions` - заявки для админа
- `PUT /api/admin/submissions/{id}` - обновление статуса

Сессии

- `POST /api/session/data` - данные сессии
- `POST /api/session/save` - сохранение сессии

Функционал

Для пользователей

- **Регистрация** по номеру телефона
- **Вход** в личный кабинет
- **Прохождение квиза** для подбора дизайна
- **Отправка заявки** на консультацию

Для администраторов

- **Доступ** по админ коду (`ADMIN123`)
- **Просмотр** всех заявок
- **Управление** статусами заявок
- **Фильтрация** и сортировка

Лицензия

MIT License - свободное использование

Быстрая проверка и запуск

```
make doctor     # инструменты, зависимости, секреты в коде, порт
make install    # закреплённые зависимости (requirements-dev.txt)
make test       # 15 тестов: аутентификация, квиз, защитные свойства
make run        # сервер на 8000 (порт задаётся переменной PORT)
make smoke-test # проверка живого инстанса
```

Документация: docs/index.md · сборка BUILD.md · матрица проверок docs/verification/matrix.md.

> **Безопасность.** Пароль приложения Gmail, который раньше был зашит в `server.py`, удалён, админ-код теперь берётся из `ADMIN_CODE` (без него админ-вход отвечает 503), CORS настраивается окружением. **Отзовите старый пароль Gmail** и задайте `ADMIN_CODE` — см. docs/security/secrets.md.

Лицензия

MIT — см. LICENSE. Использование, изменение и распространение разрешены при сохранении уведомления об авторских правах.

Документация на английском

Английская версия документации: docs/en/index.md. Каждый документ существует в двух языках и в двух форматах: `.md` и `.pdf` (`docs/pdf/ru/`, `docs/pdf/en/`). Расхождение языков ловит `make docs-check`: у каждого документа должен быть перевод и собранный PDF.

Управление проектом: `forge.sh` и `forge.bat`

Один скрипт на Unix-подобные системы (`forge.sh`) и один на Windows (`forge.bat`). Интерфейс одинаковый: те же команды, та же логика, одна реализация на каждой платформе — без bash-скриптов внутри Windows.

```
./forge.sh              # интерактивное меню
./forge.sh doctor       # проверить окружение, ничего не меняя
./forge.sh install      # поставить зависимости
./forge.sh start        # запустить в фоне, дождаться готовности, открыть браузер
./forge.sh status       # что запущено и отвечает ли порт
./forge.sh logs api -f  # логи в реальном времени
./forge.sh test         # тесты
./forge.sh stop         # остановить
```

```
forge.bat
forge.bat start
forge.bat status
```

Что умеет

Команда	Действие
`status`	состояние по портам и HTTP, а не «на глаз»
`start [--fg]`	фон (PID-файлы в `.run/`) или передний план
`stop`	останавливает дерево процессов целиком, порт освобождается
`restart`, `logs [имя] [-f]`, `open`	обычные операции
`doctor`	инструменты, версии, `.env`, `.env` в git, занятость порта
`install`, `test`, `lint`, `build`, `verify`	через `make`, если он есть
`admin`	как задать `ADMIN_CODE` (без него админка отвечает 503)

| `db migrate\|backup\|restore` | база: миграции, копия, восстановление | | `docker up\|down\|logs\|reset` | контейнеры (`down` данные не удаляет) | | `clean [--hard]` | артефакты и логи; `--hard` ещё `.venv` и `node_modules` |

Правила, заложенные в скрипт

- **Ничего не меняет без команды.** `doctor` и `status` только читают.
- **Данные не удаляются.** `stop` и `docker down` сохраняют базу; удаление — только `--hard` или явное подтверждение в `docker reset`.
- **Предполётная проверка.** Если нет `go`/python или занят порт — скрипт говорит об этом сразу, а не через две минуты таймаута.
- **Не висит после падения.** Если процесс умер, запуск прекращается и показывает последние строки лога.

- **Venv главнее системы.** Если есть `.venv`, им пользуются `make`, `pytest` и uvicorn — чтобы `install` и `test` не расходились.

Текущий способ запуска без скрипта: `python server.py` — адрес `http://localhost:8000/`.

Конфигурация: `.env`

Скрипт читает `.env` **собственным парсером**, а не через `source`: файл приходит из репозитория, а выполнять из него произвольный код нельзя. Поддержаны комментарии `#`, пустые строки, префикс `export`, одинарные и двойные кавычки, CRLF, пробелы вокруг `=` и комментарий после значения.

Приоритет значений: **переменная окружения → `.env` → встроенное значение**. Поэтому `PORT=9000 ./forge.sh start` работает независимо от файла.

Команда	Действие
`env` / `env show`	переменные, источник каждого значения и замаскированные секреты
`env init [--force]`	создать `.env` из `.env.example`: заглушки секретов заменяются случайными, права `600`, прежний файл — в `.env.bak`
`env set KEY=VALUE`	записать или заменить одну переменную (атомарно, через временный файл)
`env check`	сверка с `.env.example`: пропавшие переменные, пустые и «примерные» секреты, `.env` в `.gitignore`
`env edit`	открыть файл в `$EDITOR` (fallback: nano → vim → vi)
`env path`, `env reload`	путь к файлу и перечитывание без перезапуска

Секреты в выводе маскируются: `ADMIN_CODE=ch…de (31 симв.)`, пароль внутри строки подключения — `postgres://user:***@host`. Имена вида `*_MINUTES`, `*_URL`, `*_PORT` секретами не считаются: маскировать число минут бессмысленно.

Путь к файлу переопределяется: `FORGE_ENV_FILE=/etc/app.env ./forge.sh status`. В Windows-версии те же команды: `forge.bat env init`, `forge.bat env check`. Ограничение `cmd.exe`: значения, содержащие `%` и `!`, не поддерживаются.

Флаги и автодополнение

```
./forge.sh -h | --help            # справка
./forge.sh -V | --version         # версия скрипта, коммит, состояние .env
./forge.sh -q status              # молчать об успехах
./forge.sh --no-color status      # без ANSI (то же делает NO_COLOR=1)
./forge.sh --json status          # машинный вывод для скриптов и мониторинга

eval "$(./forge.sh completion bash)"   # автодополнение для bash
./forge.sh completion zsh > ~/.zfunc/_forge   # для zsh
```

`test`, `lint`, `build` и `verify` запускаются в окружении **без** переменных из `.env`: иначе локальные проверки вели бы себя иначе, чем CI, и тесты вида «без ключа должен быть fallback» перестали бы что-либо проверять.

Запуск в этом проекте

```
./forge.sh install && ./forge.sh start
```